PROMPT 2 — Lógica de Negocio del Backend (la verdadera)

> Documento-prompt para una IA implementadora. Debe **crear el código PHP** de la lógica de negocio del backend y entregarlo funcionando + pruebas automáticas.
>
> **Ante cualquier duda o falta de información: PÁRATE y pregunta al responsable del proyecto ANTES de escribir o modificar nada.** No asumas ni inventes.

---

1. Rol y tarea

Eres un ingeniero de software backend en PHP (PDO/MySQL).

Tarea: implementar la clase `LogicaNegocio` con los **2 métodos del diseño de la sección 3** (marcados `(new)`). Esta clase es el **único puente** entre el servidor REST y la base de datos.

2. Contexto del proyecto

- Una placa BLE emite un **iBeacon** con `uuid "EPSG-GTI-PROY-3A"` y valor `1234` **en** `minor`.
- La app Android envía por **POST** `{"minor": 1234}` (más metadatos opcionales) al servidor REST.
- El servidor REST debe guardar la medición (→ `LogicaNegocio`) y servirlas por **GET** a la web.
- La BBDD (Prompt 1) es `mediciones_db`, tabla única `mediciones` con columnas `id, uuid, major, minor, tx_power, nombre_emisora, fecha_hora`.
- **Regla de arquitectura:** el servidor REST NO toca la BBDD; todo pasa por `LogicaNegocio`.

3. Diseño a implementar (notación: Logical Design & Reverse Engineering v3)

3.1 Modelo de datos `Medicion`

```
------------------------------  Medicion  ------------------------------
|
| id             : N      (lo pone la BBDD, no llega por JSON)
| uuid           : Text
| major          : Z
| minor          : Z      (OBLIGATORIO = el valor, p.ej. 1234)
| tx_power       : Z
| nombre_emisora: Text
| fecha_hora     : Text   (la pone la BBDD, no llega por JSON)
|
-------------------------------------------------------------------------
```

3.2 Clase NUEVA `LogicaNegocio`

```
      ------------------- LogicaNegocio (NEW) -------------------
      |
      |
      |
      |
medicion: Medicion --> guardarMedicion() -->    // (new)
medicion: Medicion   <--
      |
      |
      --> recuperarMedicion() -->                // (new)
mediciones: [Medicion] <--                       //devuelve la LISTA completa
      |
      ---------------------------------------------------------------
```

3.3 Ficha de métodos

Método	Firma	Comportamiento
`guardarMedicion()`	`medicion: Medicion --> Medicion`	Valida la Medicion (ver reglas), inserta con **sentencia preparada** todos los campos en `mediciones`, y devuelve la Medicion **completa** (con `id` y `fecha_hora` reales). Lanza `InvalidArgumentException` si `minor` no es válido.
`recuperarMedicion()`	`--> mediciones: [Medicion]`	Consulta **todas** las columnas de `mediciones` con **sentencia preparada** y devuelve el array de Mediciones **de la más reciente a la más antigua** (orden por `fecha_hora` descendente, desempate por `id` descendente). Devuelve `[]` si vacío; nunca `null`.

Reglas de validación en `guardarMedicion()`:

1. **`minor`**: obligatorio, entero. Si falta o es inválido → `InvalidArgumentException` ("falta minor" / "minor inválido").
2. **Opcionales** (si vienen en el array): `major` entero, `tx_power` entero (admite negativos, es `int8`), `uuid` y `nombre_emisora` cadenas no vacías.
3. **Prohibido concatenar valores en el SQL**: siempre `prepare()` + parámetros `:minor`, `:uuid`, etc.
4. `id` y `fecha_hora` NUNCA se insertan: `fecha_hora` la pone MySQL (`CURRENT_TIMESTAMP`), `id` con `lastInsertId()`.

4. Ficheros a entregar

En `backend/logica/` y `backend/config/`:

- `LogicaNegocio.php` — clase con las 2 firmas exactas de la ficha. **No recibe la conexión por constructor**: se conecta a la BBDD llamando a `conectarBD()` (de `config.php`).
- `config/config.php` — constantes `BD_SERVIDOR`, `BD_NOMBRE`, `BD_USUARIO`, `BD_CLAVE` (`localhost`, `mediciones_db`, `root`, sin clave) y función `conectarBD(): PDO`. Las constantes se definen con

definición **condicional** (solo si no están ya definidas) para que los tests puedan apuntar a `mediciones_db_test` sin tocar la configuración real.

5. Requisitos de código y de comentarios (OBLIGATORIOS)

1. Comentarios **bien estructurados, simples, en español**.
   - Cabecera en cada fichero con qué hace y a qué objetivo del proyecto sirve.
   - Cada método con 1–2 líneas antes de la firma: qué hace, qué recibe, qué devuelve.
2. Fidelidad al estilo del proyecto: separadores `// ----`, cierre `// ()`.
3. Comentarios breves dentro del cuerpo narrando cada paso (validar / insertar / leer id / construir).

6. Pruebas automáticas (OBLIGATORIAS)

6.1 Ficheros

- `backend/tests/bootstrap.php` — define `BD_NOMBRE = "mediciones_db_test"` (antes de cargar la configuración) y crea la BBDD de prueba con el mismo esquema (Prompt 1). Así `LogicaNegocio`, al llamar a `conectarBD()`, se conecta sola a la BBDD de prueba.
- `backend/tests/LogicaNegocioTest.php` (PHPUnit) — `tearDown()` limpia con `DELETE FROM mediciones`.

6.2 Casos mínimos

1. `guardarMedicion` **guarda la Medicion de prueba** `["uuid"=>"EPSG-GTI-PROY-3A","major"=>2816,"minor"=>1234,"tx_power"=>4,"nombre_emisora"=>"Minerva_ELBACON"]` → devuelve `minor == 1234`, `id > 0`, `fecha_hora` no vacío.
2. `guardarMedicion` **lanza si** `minor` **falta o es inválido** (`0`, `-3`, no-numérico) → `InvalidArgumentException`.
3. `guardarMedicion` **acepta solo** `{"minor": 1234}` (metadatos ausentes) → guarda con `uuid NULL`.
4. `recuperarMedicion` **devuelve lo guardado** → 1 fila con `minor 1234` y el resto de campos.
5. `recuperarMedicion` **ordena descendente** con 2 filas de fechas distintas.
6. `recuperarMedicion` **devuelve** `[]` en BBDD vacía.

6.3 Ejecución

```
composer require --dev phpunit/phpunit
vendor/bin/phpunit --configuration backend/phpunit.xml backend/tests
```

Resultado: todos en verde. Documentar el comando en un bloque `/* ===== PRUEBAS ===== */`.

7. Verificación y entrega

- [ ] Solo 2 métodos públicos con las firmas exactas.
- [ ] `recuperarMedicion()` (singular) devuelve la lista, como marca el dibujo.
- [ ] Solo sentencias preparadas.
- [ ] Comentarios en español + cabeceras con explicación de para qué sirve.
- [ ] Tests en verde con un solo comando. Resumen final: ficheros y métodos añadidos.